

EXPERIMENT-3

USING EDITORS AND DEBUGGING TOOL

What is debugging?

1. Error vs Bug vs Debugging

Error = Mistake made by the programmer

E.G – Instead of $a+b$, programmer wrote $a*b$

Bug = Problem that exists in the program because of that mistake

E.g – out put will be the product not the sum

Debugging = finding and correcting the bug

2. compile time error vs runtime error vs logical error

Compile time error = An error that occurs when a compiler fails to translate a program's source code into executable machine code or byte code

Runtime error = a problem that happens while a computer program is running. It happens after the code passes the initial checks the program usually stops or freezes because it hits a task it cannot finish

Logical error = a mistake in a program's design or code that makes it run without crashing, but produce wrong or unwanted answer

3. why a program may compile but still produce wrong answers?

A program can compile successfully because its syntax and the grammar is correct and follow the rules of programming language. However it still can produce wrong answers because the compiler only checks how you wrote the code, not what you meant by it

- Types of errors in C
- Syntax Errors: These errors are basic like missing a semi colon, incorrect variable declaration, unmatched brackets

FOR EG

```
#include <stdio.h>
```

```
Int main()
```

```
{
```

```
Int x = 10;
```

```
Printf("the value of x is %d" , x)
```

```
Return 0;
```

```
}
```

The syntax error here is not putting a semi colon after the line ends

- Compilation/Type errors: When there's an error in the source code and the compiler stops the code before it runs the code

FOR Eg

```
#include <stdio.h>
```

- ```
Int main()
```

```
{
```

```
Int number = "hello world";
```

```
Printf("%d/n" , number);
```

```
Return 0;
```

```
}
```

Here the mistake is we are trying to assign a string literal to an integer variable

- Linkers error: when there's an undefined reference or missing function that links the code

FOR EG

```
#include <stdio.h>
```

```
Int calculate_total(int a , int b);
```

```
Int main()
{
Int result = calculate_total(5 , 10);
Printf("total: %d/n" , result);
Return 0;
}
```

Here we are calling a function that doesn't exist  
But the compiler assumes it is defined in other file

- Runtime error: It occurs while the program is till executing after it has successfully compiled and linked

```
#include <stdio.h>
Int main()
{
Int total_score = 100;
Int players = 0
Int average = total_score/players;
Printf("average score %d/n" , average);
Return 0;
}
```

Here we divided by zero hence its runtime error

- Logical Errors: They are flaws in a program's reasoning or algorithm that cause it to produce **incorrect or unexpected output**, even though the code compiles and runs without crashing.

For eg

```
#include <stdio.h>
```

```
Int main()
```

```
{
```

```
Int num1 = 10, num2 = 20;
```

```
Int average = num1 + num2 / 2;
```

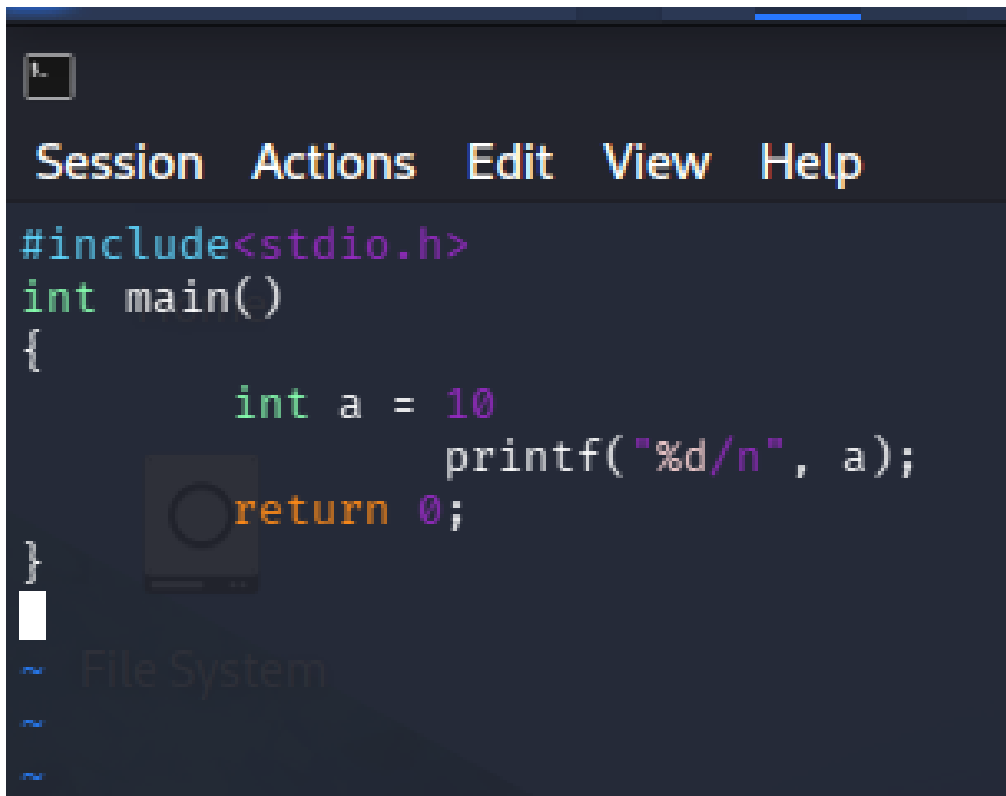
```
Printf("average: %d" , average);
```

```
Return 0;
```

```
}
```

Here it will divide 2 with 20 instead of the average giving the answer 10 instead of 20

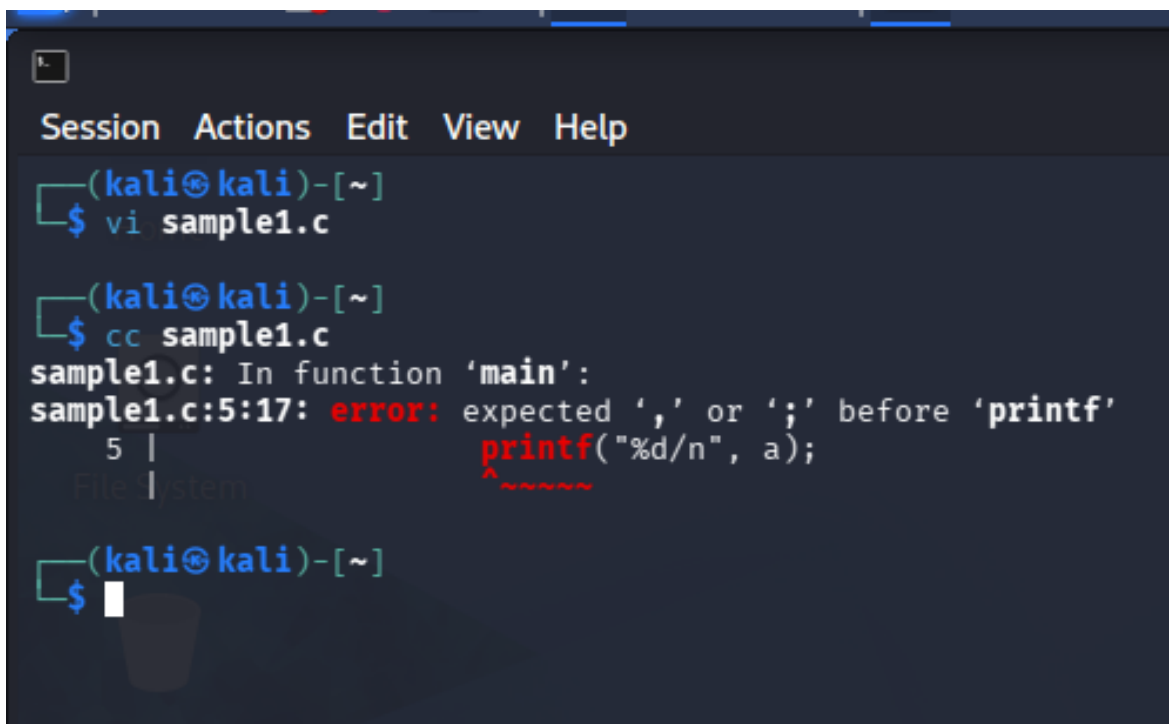
NOW THAT WE ARE DONE SEEING WHAT KIND OF ERRORS THERE ARE WE WILL DEBUG SOME CODES  
CASE-1- Syntax and compilation error



A screenshot of a code editor window. The menu bar at the top includes 'Session', 'Actions', 'Edit', 'View', and 'Help'. The code is written in C and contains a syntax error: the `printf` statement is missing a semicolon at the end. The code is as follows:

```
#include<stdio.h>
int main()
{
 int a = 10
 printf("%d/n", a);
 return 0;
}
```

The error is highlighted by a red squiggly line under the `printf` statement. A 'File System' sidebar is visible on the left.



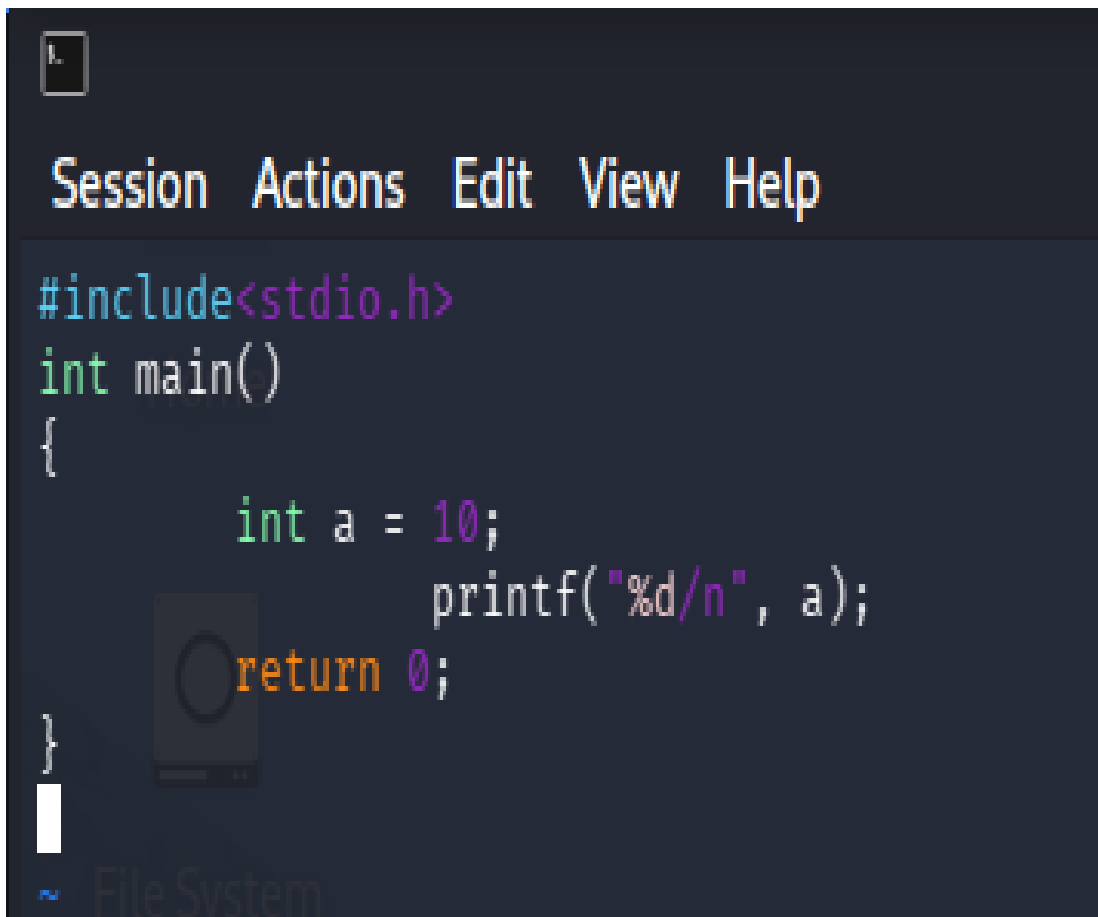
A screenshot of a terminal window showing the compilation of a C program. The prompt is `(kali㉿kali)-[~]`. The user enters `$ vi sample1.c` and then `$ cc sample1.c`. The compiler outputs the following error message:

```
sample1.c: In function 'main':
sample1.c:5:17: error: expected ',' or ';' before 'printf'
 5 | printf("%d/n", a);
 | ~~~~~
```

The error message indicates that the compiler expected a comma or semicolon before the `printf` function call. The user's prompt is `(kali㉿kali)-[~]` and the terminal shows a cursor at the end of the line.

This is an example of syntax and compilation error  
Where we didn't place an semicolon after the

Int a = 10 and the second pic depicts the compilation error where the compiler detected the bug in the code

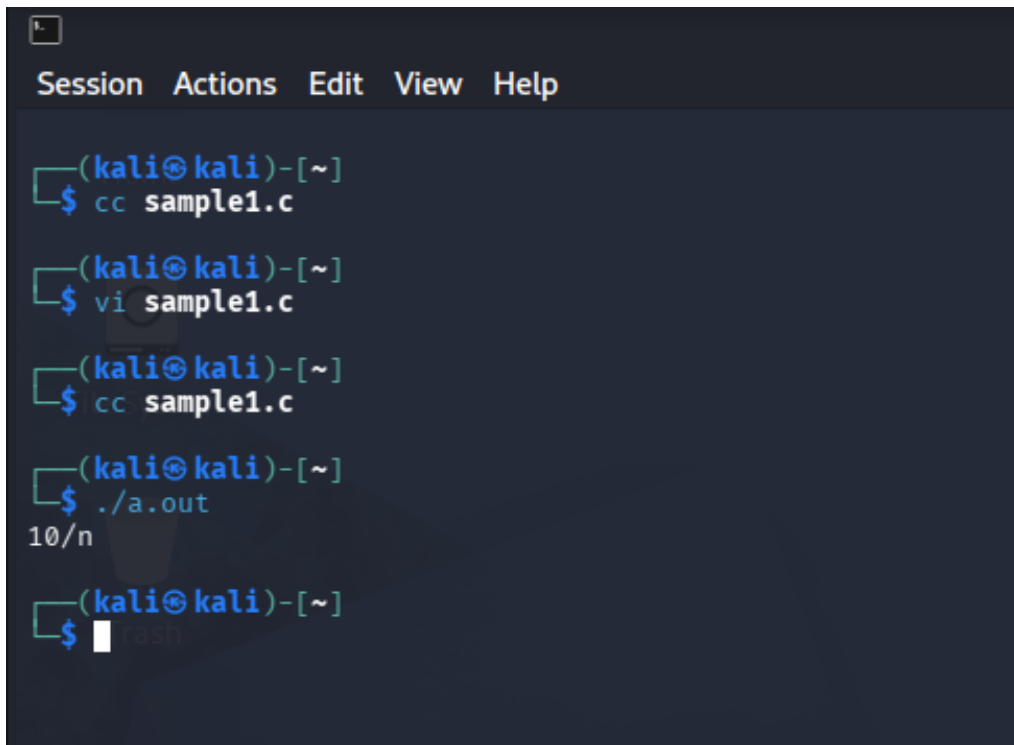


The screenshot shows a code editor window with a dark theme. The menu bar at the top includes 'Session', 'Actions', 'Edit', 'View', and 'Help'. The code being edited is a C program that includes `<stdio.h>` and defines a `main` function. Inside the function, an integer `a` is initialized to 10, and `printf` is used to print its value. The program ends with `return 0;`. A compilation error is visible, indicated by a red squiggly line under the `return 0;` statement. The error message, partially obscured by a dark overlay, reads: 'error: incompatible integer to char conversion: '0' (aka '0') to '0' (aka '0')'. The bottom status bar shows a file system icon and the text 'File System'.

```
#include<stdio.h>
int main()
{
 int a = 10;
 printf("%d/n", a);
 return 0;
}
```

error: incompatible integer to char conversion: '0' (aka '0') to '0' (aka '0')

File System



```
Session Actions Edit View Help

(kali㉿kali)-[~]
$ cc sample1.c

(kali㉿kali)-[~]
$ vi sample1.c

(kali㉿kali)-[~]
$ cc sample1.c

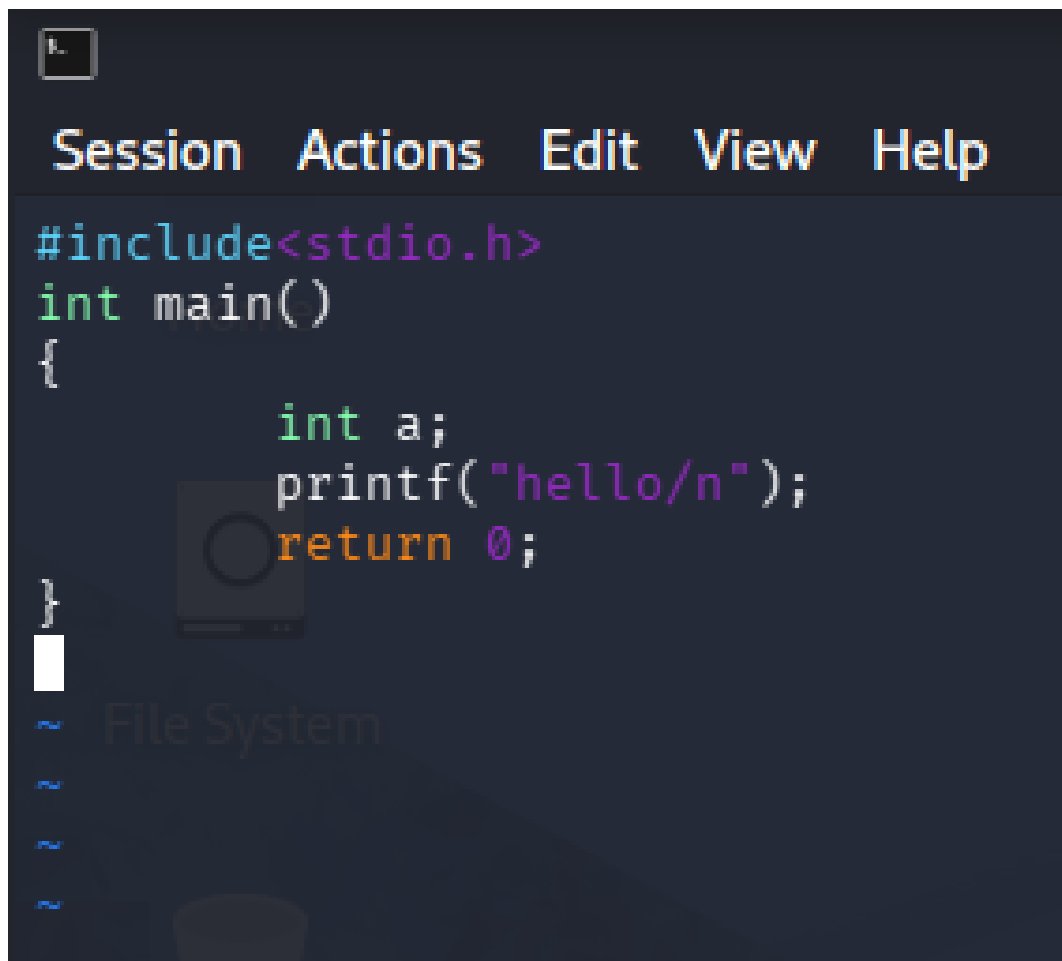
(kali㉿kali)-[~]
$./a.out
10/n

(kali㉿kali)-[~]
$
```

In the above to pics we have debugged the code and it is running how it is supposed to

CASE-2 Compiler warning (-Wall , -Wextra)

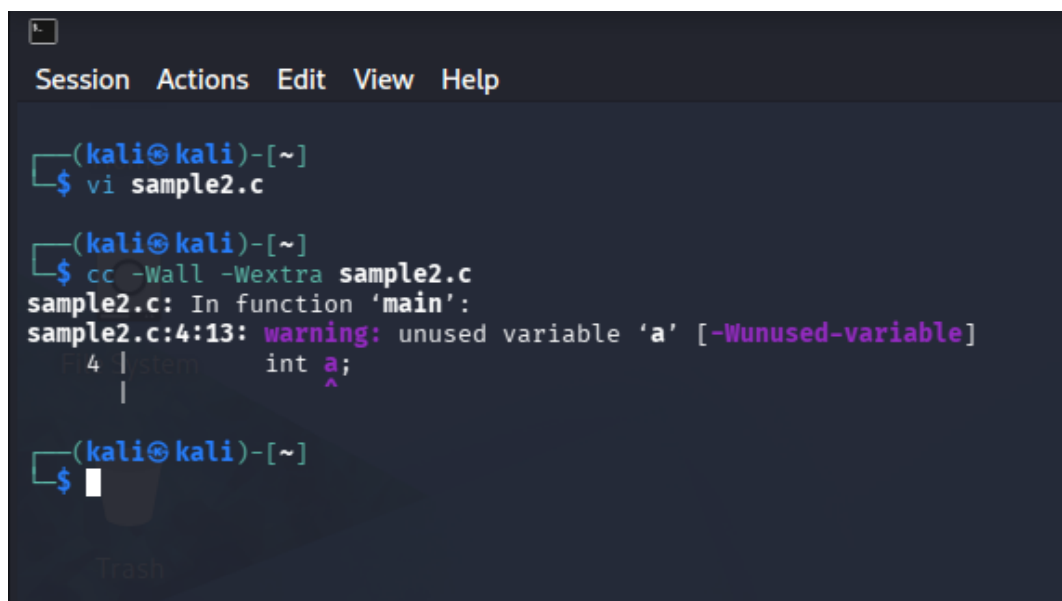




A screenshot of a Vim text editor window. The title bar at the top shows a small icon on the left and the text "Session Actions Edit View Help" on the right. The main editing area contains a C program with the following code:

```
#include<stdio.h>
int main()
{
 int a;
 printf("hello/n");
 return 0;
}
```

The code is color-coded: preprocessor directives are purple, keywords are green, identifiers are white, and literals are orange. A white cursor is positioned at the end of the closing brace on the sixth line. On the left side of the editor, a "File System" sidebar is visible, showing a tree structure with a tilde (~) at the top and several sub-items below it.



A screenshot of a terminal window with a dark background. The prompt is "(kali㉿kali)-[~]". The user enters the command "\$ vi sample2.c", which opens the file in Vim. The user then enters the command "\$ cc -Wall -Wextra sample2.c" to compile the program. The terminal output shows the compilation process and a warning:

```
sample2.c: In function 'main':
sample2.c:4:13: warning: unused variable 'a' [-Wunused-variable]
 4 | int a;
 | ^
```

The warning message is color-coded, with "warning:" in purple and the variable name 'a' in orange. The user then enters the command "\$" at the prompt, and the cursor is positioned at the end of the line. At the bottom of the terminal window, a "Trash" icon is visible.

This code will run if we compile it normally with the command `cc sample2.c` but if we use the command `cc -Wall -Wextra sample2.c` it will show an unused variable

### CASE-3 Printf based debugging (Logical bug)



```
Session Actions Edit View Help
#include<stdio.h>
int main(void)
{
 int i, sum = 0;
 for(i = 1; i < 5; i++)
 {
 printf("DEBUG: i = %d/n", i);
 sum = sum + i;
 printf("DEBUG: sum = %d/n", sum);
 }
 return 0;
}
```

```
kali@kali: ~
Session Actions Edit View Help

(kali@kali)-[~]
$ vi sample5.c

(kali@kali)-[~]
$ cc sample5.c

(kali@kali)-[~]
$./a.out
DEBUG: i = 1/nDEBUG: sum = 1/nDEBUG: i = 2/nDEBUG: sum = 3/nDEBUG: i = 3/nDEBUG: sum = 6/nDEBUG: i = 4/nDEBUG: sum = 10/n

(kali@kali)-[~]
$
```

In this code there is a logical error the code runs fine but we need an answer for the sum of first five numbers i.e 1-5 but the code gives us answer to 1-4 to fix it we will just add  $i \leq 5$  in the source code

```
Session Actions Edit View Help

#include<stdio.h>
int main(void)
{
 int i, sum = 0;
 for(i = 1; i ≤ 5; i++)
 {
 printf("DEBUG: i = %d/n", i);
 sum = sum + i;
 printf("DEBUG: sum = %d/n", sum);
 }
 return 0;
}
```

```
Session Actions Edit View Help

(kali@kali)-[~]
$ vi sample5.c

(kali@kali)-[~]
$ cc sample5.c

(kali@kali)-[~]
$./a.out
DEBUG: i = 1/nDEBUG: sum = 1/nDEBUG: i = 2/nDEBUG: sum = 3/nDEBUG: i = 3/nDEBUG: sum = 6/nDEBUG: i = 4/nDEBUG: sum = 10/n

(kali@kali)-[~]
$ vi sample5.c

(kali@kali)-[~]
$ cc sample5.c

(kali@kali)-[~]
$./a.out
DEBUG: i = 1/nDEBUG: sum = 1/nDEBUG: i = 2/nDEBUG: sum = 3/nDEBUG: i = 3/nDEBUG: sum = 6/nDEBUG: i = 4/nDEBUG: sum = 10/nDEBUG: i = 5/nDEBUG: sum = 15/n

(kali@kali)-[~]
$ vi sample5.c

(kali@kali)-[~]
$
```

NOW WE WILL USE GDB TO DEBUG THE CODE

## CASE-4 Debugging a logical error using GDB

```
1 #include<stdio.h>
2 int main()
3 {
4 int i, sum = 0
5 for(i = 1; i < 5; i++)
6 {
7 sum = sum + i;
8 }
9 printf("sum = %d/n", sum);
10 return 0;
11 }
12 |
```

```
Session Actions Edit View Help
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
 <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word" ...
Reading symbols from ./sample...
(gdb) list
1 #include <stdio.h>
2 int main()
3 {
4 int i, sum = 0;
5 for (i = 1; i < 5; i++)
6 {
7 sum = sum + i;
8 }
9 printf("sum = %d\n", sum);
10 return 0;
(gdb) break 7
Breakpoint 1 at 0x1151: file sample.c, line 7.
(gdb) run
Starting program: /home/kali/sample
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, main () at sample.c:7
7 sum = sum + i;
(gdb) print i
$1 = 1
(gdb) display i
1: i = 1
(gdb) display sum
2: sum = 0
(gdb) continue
Continuing.

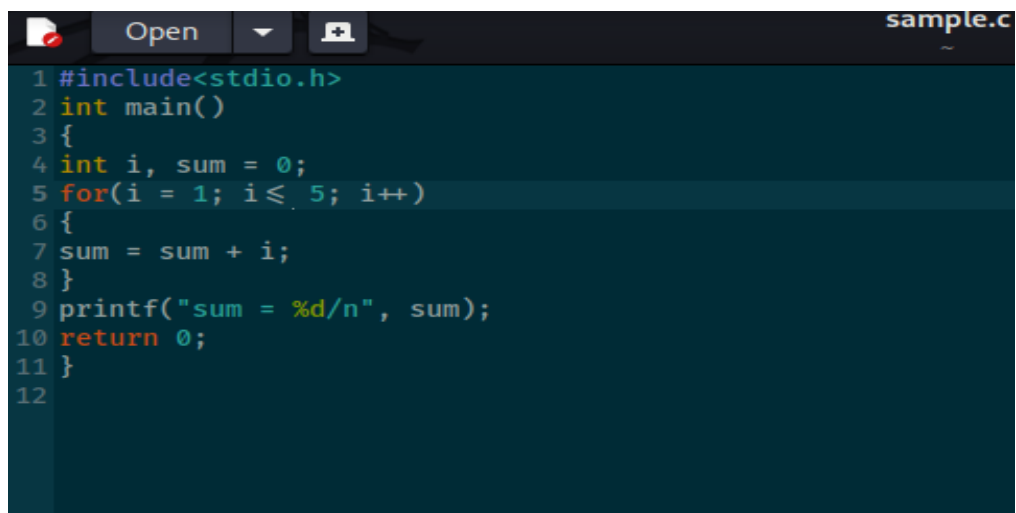
Breakpoint 1, main () at sample.c:7
7 sum = sum + i;
1: i = 2
2: sum = 1
(gdb) continue
Continuing.

Breakpoint 1, main () at sample.c:7
7 sum = sum + i;
1: i = 3
2: sum = 3
(gdb) continue
Continuing.

Breakpoint 1, main () at sample.c:7
7 sum = sum + i;
1: i = 4
2: sum = 6
(gdb) continue
Continuing.
sum = 10/n[Inferior 1 (process 25440) exited normally]
(gdb) |
```

In this code the logical error is same as the case-3 but in this we debugged with the help of GDB and we wrote the code in gedit instead of VI

If we fix the source code it will be like this



The image shows a code editor window with a dark theme. The title bar at the top includes a file icon, an 'Open' button, a dropdown arrow, and a 'sample.c' label. The code is written in C and calculates the sum of integers from 1 to 5. The code is as follows:

```
1 #include<stdio.h>
2 int main()
3 {
4 int i, sum = 0;
5 for(i = 1; i ≤ 5; i++)
6 {
7 sum = sum + i;
8 }
9 printf("sum = %d/n", sum);
10 return 0;
11 }
12
```

```
Session Actions Edit View Help
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./sample...
(gdb) list
1 #include<stdio.h>
2 int main()
3 {
4 int i, sum = 0;
5 for(i = 1; i <= 5; i++)
6 {
7 sum = sum + i;
8 }
9 printf("sum = %d\n", sum);
10 return 0;
(gdb) break 7
Breakpoint 1 at 0x1151: file sample.c, line 7.
(gdb) run
Starting program: /home/kali/sample
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, main () at sample.c:7
7 sum = sum + i;
(gdb) print i
$1 = 1
(gdb) display sum
1: sum = 0
(gdb) continue
Continuing.

Breakpoint 1, main () at sample.c:7
7 sum = sum + i;
1: sum = 1
(gdb) continue
Continuing.

Breakpoint 1, main () at sample.c:7
7 sum = sum + i;
1: sum = 3
(gdb) continue
Continuing.

Breakpoint 1, main () at sample.c:7
7 sum = sum + i;
1: sum = 6
(gdb) continue
Continuing.

Breakpoint 1, main () at sample.c:7
7 sum = sum + i;
1: sum = 10
(gdb) continue
Continuing.
sum = 15/n[Inferior 1 (process 29586) exited normally]
(gdb) █
```

Now the answer is correct

## CASE-5 Debugging functions (next vs step)

In this we will debug using two steps

1) next

2) step

By using next to debug

```
1 #include <stdio.h>
2 int square(int x)
3 {
4 int result1;
5 result1 = x * x;
6 return result1;
7 }
8 int main()
9 {
10 int result, n = 5;
11 result = square(n);
12 printf("square=%d/n", result);
13 return 0;
14 }
```



```
Session Actions Edit View Help
GNU gdb (Debian 17.2-1+b1) 17.2
Copyright (C) 2025 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./sample6...
(gdb) list
4 int result1;
5 result1 = x * x;
6 return result1;
7 }
8 int main()
9 {
10 int result, n = 5;
11 result = square(n);
12 printf("square=%d/n", result);
13 return 0;
(gdb) break main
Breakpoint 1 at 0x1156: file sample6.c, line 10.
(gdb) run
Starting program: /home/kali/sample6
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, main () at sample6.c:10
10 int result, n = 5;
(gdb) next
11 result = square(n);
(gdb) next
12 printf("square=%d/n", result);
(gdb) print result
$1 = 25
(gdb) quit
A debugging session is active.

 Inferior 1 [process 37276] will be killed.

Quit anyway? (y or n) y
```

Now we will use step to debug

```

(kali㉿kali)-[~]
$ gedit sample6.c

** (gedit:39110): WARNING **: 13:18:45.529: Could not load Peas repository: Typelib file for namespace 'Peas', version '1.0' not found
** (gedit:39110): WARNING **: 13:18:45.529: Could not load PeasGtk repository: Typelib file for namespace 'PeasGtk', version '1.0' not found

(kali㉿kali)-[~]
$ gdb ./sample6
GNU gdb (Debian 17.2-1+b1) 17.2
Copyright (C) 2025 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word" ...
Reading symbols from ./sample6 ...
(gdb) break main
Breakpoint 1 at 0x1156: file sample6.c, line 10.
(gdb) run
Starting program: /home/kali/sample6
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, main () at sample6.c:10
10 int result, n = 5;
(gdb) next
11 result = square(n);
(gdb) step
square (x=5) at sample6.c:5
5 result1 = x * x;
(gdb) print x
$1 = 5
(gdb) next
6 return result1;
(gdb) print result
X No symbol "result" in current context.
(gdb) print result1
$2 = 25
(gdb) █

```

## CASE-6 Debugging runtime errors using backtrace (backtrace / bt)

```
sample7.c
1 #include <stdio.h>
2 int divide(int a, int b)
3 {
4 return a / b;
5 }
6 int calculate(int x)
7 {
8 return divide(x, 0);
9 }
10 int main()
11 {
12 int result = calculate(10);
13 printf("result:%d/n", result);
14 return 0;
15 }
```

```
Session Actions Edit View Help
└─$ cc -g sample7.c -o sample7

(kali@kali)~$
└─$ gdb ./sample7
GNU gdb (Debian 17.2-1+b1) 17.2
Copyright (C) 2025 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word" ...
Reading symbols from ./sample7...
(gdb) run
Starting program: /home/kali/sample7
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".

Program received signal SIGFPE, Arithmetic exception.
0x0000555555555147 in divide (a=10, b=0) at sample7.c:4
4 return a / b;
(gdb) backtrace
#0 0x0000555555555147 in divide (a=10, b=0) at sample7.c:4
#1 0x0000555555555166 in calculate (x=10) at sample7.c:8
#2 0x000055555555517a in main () at sample7.c:12
(gdb) frame 0
#0 0x0000555555555147 in divide (a=10, b=0) at sample7.c:4
4 return a / b;
(gdb) print b
$1 = 0
(gdb) frame 1
#1 0x0000555555555166 in calculate (x=10) at sample7.c:8
8 return divide(x, 0);
(gdb) print x
$2 = 10
(gdb) quit
```

## CASE-7 Debugging array out-of-bounds error using GDB

```
kali@kali: ~
Session Actions Edit View Help
(kali@kali)-[~]
$ gedit sample8.c
** (gedit:59967): WARNING **: 13:56:32.607: Could not load Peas repository: Typelib file for namespace 'Peas', version '1.0' not found
** (gedit:59967): WARNING **: 13:56:32.607: Could not load PeasGtk repository: Typelib file for namespace 'PeasGtk', version '1.0' not found
(kali@kali)-[~]
$ cc -g sample8.c -o sample8
(kali@kali)-[~]
$ gdb ./sample8
GNU gdb (Debian 17.2-1+b1) 17.2
Copyright (C) 2025 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
 <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word" ...
Reading symbols from ./sample8 ...
(gdb) break 8
Breakpoint 1 at 0x114a: file sample8.c, line 8.
(gdb) run
Starting program: /home/kali/sample8
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, main () at sample8.c:8
8 scanf("%d\n", &a[i]);
(gdb) run
The program being debugged has been started already.
Start it from the beginning? (y or n) n
X Program not restarted.
(gdb) print &a[0]
$1 = (int *) 0x7fffffffdd20
(gdb) print &a[4]
$2 = (int *) 0x7fffffffdd30
(gdb) print &a[5]
$3 = (int *) 0x7fffffffdd34
(gdb) █
```

## CASE-8 Segmentation faults (SIGSEGV) Using GDB

```
sample9.c
#include <stdio.h>
int main()
{
 int *p = NULL;
 *p = 10;
 printf("%d\n", *p);
 return 0;
}
```

```
(kali@kali)-[~]
$ gdb ./sample9
GNU gdb (Debian 17.2-1+b1) 17.2
Copyright (C) 2025 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word" ...
Reading symbols from ./sample9...
(gdb) run
Starting program: /home/kali/sample9
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".

Program received signal SIGSEGV, Segmentation fault.
0x000055555555514d in main () at sample9.c:5
warning: Source file is more recent than executable.
5 int *p = &x;
(gdb) print p
$1 = (int *) 0x0
(gdb) prit *p
X Undefined command: "prit". Try "help".
(gdb) print *p
X Cannot access memory at address 0x0
(gdb) |
```

This before we wrote the source code correctly

```
1 #include <stdio.h>
2 int main()
3 {
4 int x;
5 int *p = &x;
6 *p = 10;
7 printf("%d/n", *P);
8 return 0;
9 }
10
```

```

Session Actions Edit View Help
** (gedit:64542): WARNING **: 14:05:05.666: Could not load Peas repository: Typelib file for namespace 'Peas', version '1.0' not found
** (gedit:64542): WARNING **: 14:05:05.666: Could not load PeasGtk repository: Typelib file for namespace 'PeasGtk', version '1.0' not found

(kali@kali)-[~]
$ cc -g sample9.c -o sample9

(kali@kali)-[~]
$ gdb ./sample9
GNU gdb (Debian 17.2-1+b1) 17.2
Copyright (C) 2025 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./sample9...
(gdb) run
Starting program: /home/kali/sample9
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".

Program received signal SIGSEGV, Segmentation fault.
0x000055555555514d in main () at sample9.c:5
5 *p = 10;
(gdb) print p
$1 = (int *) 0x0

```

IN this experiment we learned how to debug source code via multiple methods like GDB by printf and many more.